

Devpost submission — ready to paste

Track: **Showcase** · lead category: **Potential Impact**.

Submission name

Shockwave — turn GitLab Orbit's graph into a merge decision

Elevator pitch (tagline)

Code review's hardest question — "if I change this, what breaks?" — finally answered. Shockwave reads GitLab

Try it out (links)

Live site & Blast Monitor: <https://shockwave-ochw.vercel.app/>
Code (MIT, public): <https://gitlab.com/uthmannabeel-group/Shockwave-project>
AI Catalog agent: <https://gitlab.com/explore/ai-catalog/agents/1011457/>
Live MR-bot review: https://gitlab.com/uthmannabeel-group/Shockwave-project/-/merge_requests/1

Demo video

<paste your YouTube/Vimeo link>

Built With

python, gitlab-orbit, gitlab-duo, gitlab-ai-catalog, gitlab-ci, flask, duckdb, d3.js, javascript, vercel, re

Project story (paste into "Tell us about your project")

The one question code review can't answer

Ship a one-line change. Three files away, production breaks. Nobody caught it — because the reviewer was reading a **diff**, and the answer lived in the **call graph**. `grep` finds the word; it can't follow a cross-file, inherited, resolved reference. So teams approve on hope.

GitLab Orbit just turned the codebase into a queryable knowledge graph. **Shockwave is the layer that turns that graph into a decision.**

What it does — review *with the answer*, not a guess

Point it at a function, a file, or a whole merge-request diff. In seconds, Shockwave returns:

- 🚦 **A risk verdict** — `LOW / REVIEW / HIGH`, that the merge-request bot can gate on.
- ⚡ **The blast radius** — every definition that transitively depends on the change, across files, ranked by fan-in × proximity.

-  **Exposure** — is the change reachable from a public route, API, or CLI? With the **exact call path**. A break the world can see, versus one it can't.
-  **The tests that matter** — not your whole suite, the *specific* tests that exercise this change, as a copy-paste `pytest` command.
-  **The tests you're missing** — high-impact code no test touches, each with a generated stub.
-  **What it depends on** — the reverse view, so review is complete in both directions.

One Flask decorator. 114 definitions across 10 files. Reachable from `@route`. 58 tests that actually matter — none of which `grep` could have found.

Not a dashboard you have to remember to open

Five surfaces, one graph: a **CLI**, an autonomous **merge-request bot** that posts the review automatically, a **GitLab Duo agent** published in the AI Catalog, an interactive **Blast Monitor** (a live impact map), and a reusable **Orbit Reachability skill** so any agent can query Orbit correctly. It meets developers exactly where review already happens.

The feature big companies build whole products around

"Which tests should I run for this change?" — Google's Test Impact Analysis and startups like Launchable are entire businesses answering it. Shockwave gets it **for free** from the graph and puts it on every merge request. That alone changes CI economics.

How we built it

A small, tested Python engine with **two Orbit backends**: **Orbit Local** (a DuckDB graph via `orbit sql`) and **Orbit Remote** (the hosted graph over its JSON traversal DSL). Remote forbids full-graph scans, so the transitive radius is an **iterative anchored BFS** — one query per hop, cycle-safe, path-reconstructing. On top: a GitLab CI bot that posts and updates the MR review, a Duo agent driving Orbit's `query_graph` tool, a D3 web app, and a static export on Vercel (with real baked analyses so the demo runs with zero backend).

What makes it real — and honest

Two **genuine** Orbit integrations, not a mock. The MR review on our own repo is **computed live from the graph**. 23 passing tests. And — rare for a hackathon — Shockwave is **explicit about what it can't see** and flags its own caveats (partial results, ambiguous names, the indexed commit), because a reviewer that overstates is worse than no reviewer at all.

Challenges we turned into wins

- **Orbit Local ≠ Remote**. Local flattens code edges into `gl_edge` with no `ImportedSymbol→Definition` link, so cross-file calls are bridged by name and disambiguated by `import_path`; Remote resolves them but rejects unbounded queries — which is exactly why we built the anchored hop-by-hop walk.
- **Accuracy honesty**. "Untested" overstated a heuristic that only checks *direct* test callers — we reworded it and replaced naïve substring matching with word-boundary matching to kill false positives (`latest.py` is no longer "a test").
- **Security reachability** isn't Orbit-native yet: the `Finding` node carries no code location, so it needs a join with GitLab's Vulnerability API — that's our headline roadmap item, with the engine already built.

What's next

- **Security reachability triage** — point the same engine at vulnerabilities: rank them by whether they're *actually reachable* from an entry point. Alert-fatigue, solved by the graph.
- Line-level diff seeding, more edge types and languages, and an IDE gutter that shows a symbol's blast radius as you type.

The 30-second pitch: *grep* tells you where a name appears. *Shockwave* tells you what a change breaks, whether the world can see it, and exactly which tests to run — and posts it on every merge request, straight from GitLab Orbit.